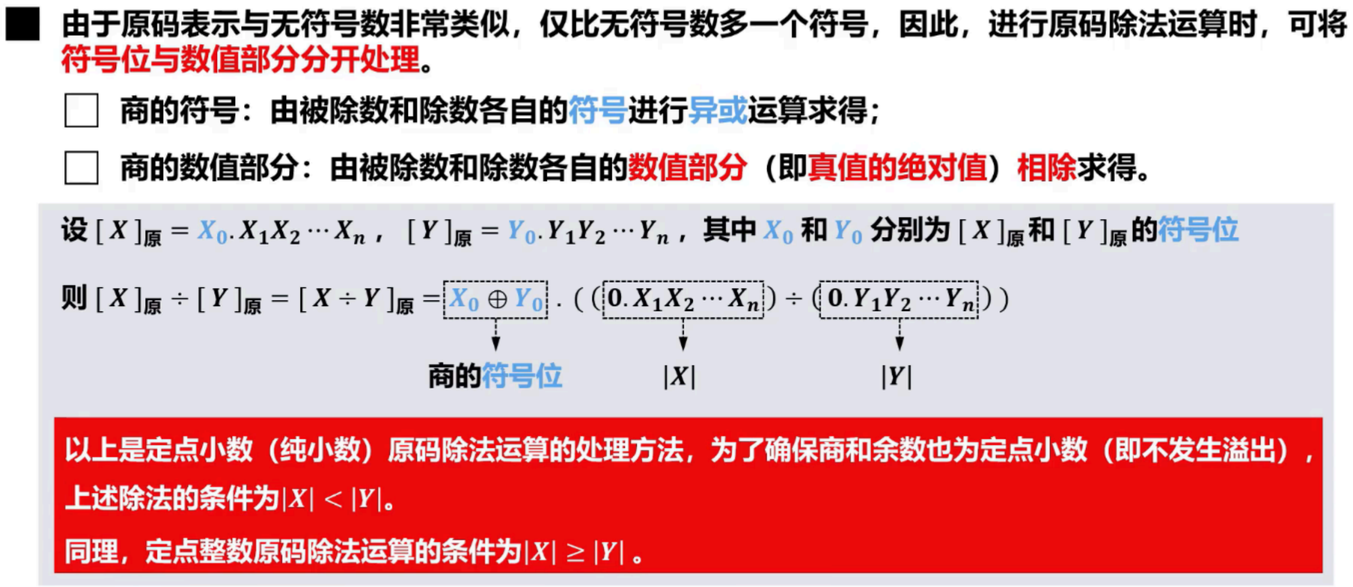
一、法则



■ 由于原码表示与无符号数非常类似，仅比无符号数多一个符号，因此，进行原码除法运算时，可将**符号位与数值部分分开处理**。

- □ 商的符号：由被除数和除数各自的**符号**进行**异或**运算求得；
- □ 商的数值部分：由被除数和除数各自的**数值部分**（即**真值的绝对值**）**相除**求得。

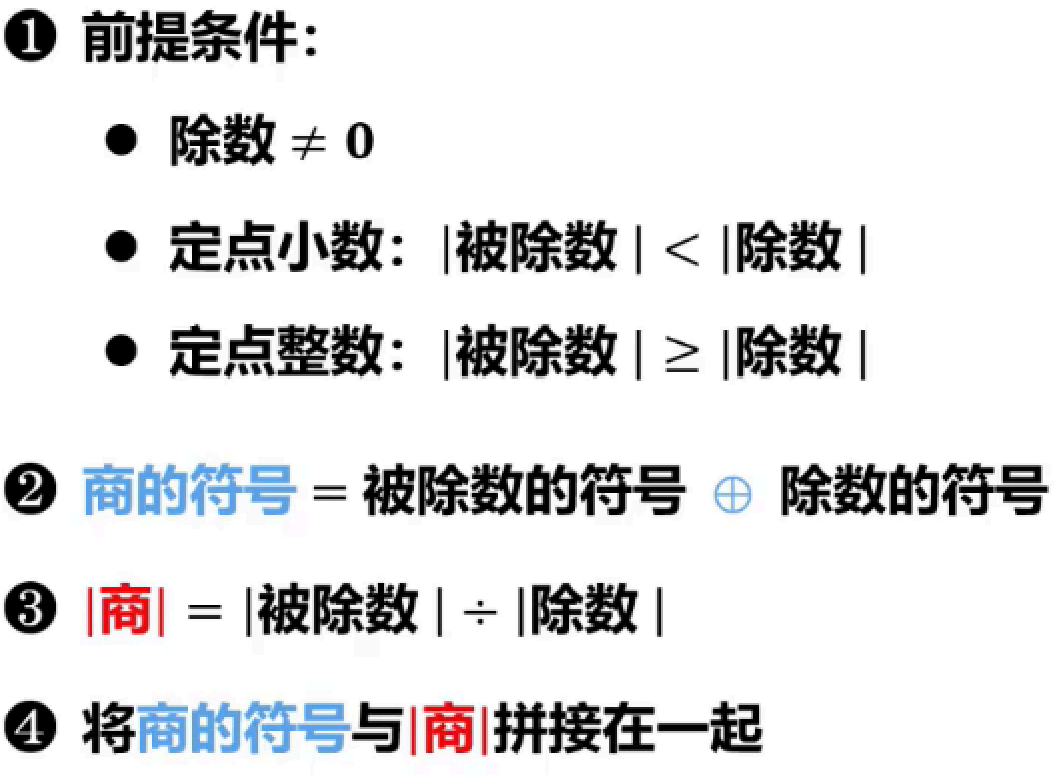
设 $[X]_原 = X_0.X_1X_2\cdots X_n$，$[Y]_原 = Y_0.Y_1Y_2\cdots Y_n$，其中 X_0 和 Y_0 分别为 $[X]_原$ 和 $[Y]_原$ 的符号位

$$\text{则 } [X]_原 \div [Y]_原 = [X \div Y]_原 = \underbrace{\boxed{X_0 \oplus Y_0}}_{\text{商的符号位}}.\left(\left(\underbrace{\boxed{0.X_1X_2\cdots X_n}}_{|X|}\right) \div \left(\underbrace{\boxed{0.Y_1Y_2\cdots Y_n}}_{|Y|}\right)\right)$$

以上是定点小数（纯小数）原码除法运算的处理方法，为了确保商和余数也为定点小数（即不发生溢出），上述除法的条件为$|X| < |Y|$。

同理，定点整数原码除法运算的条件为$|X| \geq |Y|$ 。

简单而言



❶ 前提条件：

- 除数 $\neq 0$
- 定点小数：$|$被除数$| < |$除数$|$
- 定点整数：$|$被除数$| \geq |$除数$|$

❷ 商的符号 = 被除数的符号 $\oplus$ 除数的符号

❸ $|$商$| = |$被除数$| \div |$除数$|$

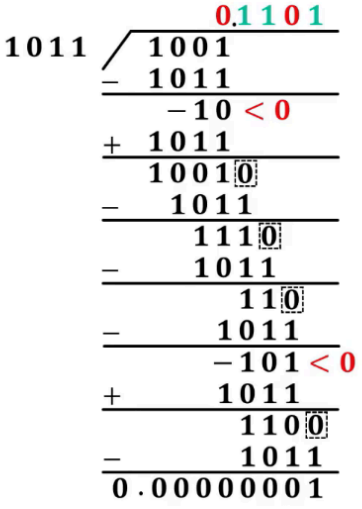
❹ 将商的符号与$|$商$|$拼接在一起

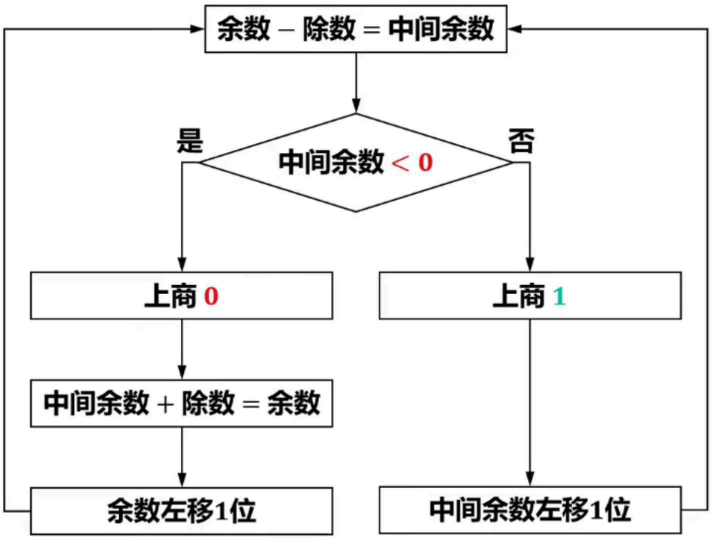
二、计算机底层的除法运算逻辑

默认够减，依据结果正负，来确定上商数字以及是否恢复余数

模拟二进制除法笔算过程

计算机无法直接判断大小，只能首先默认“够减”





三、举例

除数	被除数、余数	商
X	ACC	MQ

【举例】 设$[x]_{原}= 1.1001$，$[y]_{原}= 0.1011$，用恢复余数法求 $[x]_{原} \div [y]_{原}$。

解析：

$[|x|]_{补} = 0.1001$

$[|y|]_{补} = 0.1011$

$[-|y|]_{补} = 1.0101$

商的符号 = x 的符号 $\oplus$ y 的符号

$= 1 \oplus 0 = 1$

$[x]_{原} \div [y]_{原} = 1.1101$

余数$R = 0.0001 \times 2^{-4}$

	余数R	商Q	
	0 . 1 0 0 1	0 . 0 0 0 0	初始余数R_0就是被除数x
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	1 . 1 1 1 0	0 . 0 0 0 0	$R_1 < 0$，上商0
$+[\|y\|]_{补}$	0 . 1 0 1 1		加上除数y，恢复余数
	0 . 1 0 0 1		
	1 . 0 0 1 0	0 . 0 0 0	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	0 . 0 1 1 1	0 . 0 0 0 1	$R_2 > 0$，上商1
	0 . 1 1 1 0	0 . 0 0 1	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	0 . 0 0 1 1	0 . 0 0 1 1	$R_3 > 0$，上商1
	0 . 0 1 1 0	0 . 0 1 1	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	1 . 1 0 1 1	0 . 0 1 1 0	$R_4 < 0$，上商0
$+[\|y\|]_{补}$	0 . 1 0 1 1		加上除数y，恢复余数
	0 . 0 1 1 0		
	0 . 1 1 0 0	0 . 1 1 0	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	0 . 0 0 0 1	0 . 1 1 0 1	$R_5 > 0$，上商1

三、恢复余数法的缺点

从本例可以看出，由于运算过程中可能需要恢复余数，并且恢复余数的次数和具体的操作数相关，极端情况下每一步运算都需要恢复余数，因此，该方法的最大缺点是运算时间不确定，另外，其控制电路也比较复杂。